The Fast Typist, Not the Strategist: What 23 Days of Human-AI Mathematical Collaboration Reveals About LLMs and Formal Reasoning

Jason Robert Gochanour

April 20, 2026

Abstract

We report on a 23-day collaboration between a mathematician and an LLM-based AI coding assistant to formalize the Nyman–Beurling criterion for the Riemann Hypothesis in Lean 4. The resulting artifact—the *Cathedral*—comprises 22,670 lines of machine-verified proof across 84 files, with 641 theorems and 40 axioms. We analyze the division of labor between human and AI, finding a genuinely **collaborative** dynamic: the AI generated ∼85% of code by volume and actively proposed proof strategies, designed axiom elimination campaigns, identified algebraic shortcuts, and recognized failing approaches—while the human provided architectural vision, mathematical direction, and quality control. We quantify the AI's productivity gains (∼5× throughput vs. solo human), identify four failure modes specific to LLM-assisted theorem proving (stale context, API hallucination, axiom drift, over-eagerness), and compare with existing AI-for-math systems (AlphaProof, Lean Copilot, LeanDojo). We argue that the collaboration reveals a *complementary* partnership where the human provides taste and direction while the AI provides velocity *and* substantive mathematical contributions.

Contents

1 Introduction

Can AI prove theorems? The question has animated research since Newell and Simon's Logic Theorist (1956) and has recently gained momentum with AlphaProof's IMO silver medal (2024), Lean Copilot (2024), and LeanDojo's retrieval-augmented proving (2023).

These systems operate at the level of individual lemmas: given a statement, find its proof. The Cathedral project operates at a qualitatively different scale: not proving one theorem, but designing an entire proof *architecture*—84 files, 11 modules, 644 theorems—for a major open problem in mathematics.

This paper analyzes the human-AI collaboration that produced the Cathedral, focusing on the question: *what did the AI actually do?*

1.1 Summary of Findings

1. The AI generated ∼85% of all Lean code by volume and actively proposed proof strategies that were adopted.

2. The AI's value was in both **throughput** and **mathematical reasoning**: it proposed axiom elimination paths, designed proof architectures for individual modules, and identified when approaches were failing.

3. The human's irreplaceable contribution was **mathematical taste**: the final decision on proof aesthetics, axiom priorities, and the overall architectural vision that unified 11 modules into a coherent whole.

4. The collaboration produced a 54% discard rate, higher than either human or AI would produce alone, suggesting that the pairing enables more aggressive exploration.

2 System and Methodology

2.1 The AI System

The AI assistant was an LLM-based coding agent with the following capabilities:

- File reading and writing within the project workspace.
- Terminal command execution (`lake build`, `lake env lean`).
- Web search for Mathlib documentation and API references.
- Context window of ∼200K tokens (insufficient for the full 20,000-line codebase).
- No persistent memory between sessions (context rebuilt each time).

The AI had no special-purpose theorem-proving capabilities. It was a general-purpose coding assistant that happened to be writing Lean 4.

2.2 The Human

The human collaborator is a mathematician with expertise in analytic number theory and experience with formal verification. The human provided:

- Mathematical strategy and proof architecture.
- Axiom classification and elimination priorities.
- Quality control (deciding when a proof was "done" vs. "correct but ugly").
- Error diagnosis when the AI's approach failed.

2.3 Session Protocol

Development occurred in sessions of 2–8 hours over 23 days. A typical session:

1. **Human sets goal**: "Eliminate axiom X" or "Prove theorem Y."
2. **AI reads context**: Relevant source files, dependencies, Mathlib API.
3. **AI proposes strategy**: "I'll use Mathlib's `riemannZeta_ne_zero_of_one_le_re` and the functional equation."
4. **Human approves or redirects**: "Good approach" or "No, try the contrapositive via the Mellin transform instead."
5. **AI writes code**: Full Lean proof, typically 50–300 lines.
6. **Compiler checks**: AI runs `lake env lean` and iterates on errors.
7. **Regression**: AI runs `lake build` to check full-codebase consistency.
8. **Human reviews**: Checks mathematical content, docstrings, axiom impact.

3 Division of Labor: Quantitative Analysis

3.1 Task Taxonomy

We classify all development tasks into six categories:

Category	Example	Human	AI
Overall strategy	"Use Nyman–Beurling, not Robin"	80%	20%
Proof strategy	"Use Rank-1 Mellin for converse"	60%	40%
Architecture	"Create 11-module structure"	70%	30%
Implementation	"Write the Lean proof"	15%	85%
Maintenance	"Update docstrings, fix imports"	5%	95%
Debugging	"Why does this type not unify?"	40%	60%

3.2 Volume Analysis

Metric	Value
Total LOC produced (active + archived)	43,041
Estimated AI-authored LOC	∼36,500 (85%)
Estimated human-authored LOC	∼6,500 (15%)
Files created	174
Files archived (discarded)	96 (55%)
Axioms declared	∼80 (over lifetime)
Axioms eliminated	40
Axiom elimination proofs	40
Of which AI-designed	∼35 (88%)

A subtlety in attribution: while the human set the axiom elimination *priorities*, the AI designed and executed the vast majority of elimination *proofs*—identifying which Mathlib lemmas to chain, proposing algebraic strategies (e.g., avoiding topological filter arguments in favor of direct bounds), and recognizing when a proof path was blocked. The AI also proposed several proof strategies that were adopted: the Abel summation architecture, the floor-sum telescoping approach, and the Rank-1 factorization technique for the Mellin converse. The overall vision was human-directed, but the mathematical contributions were genuinely collaborative.

3.3 Throughput Analysis

Estimated time comparison for key tasks:

Task	Human solo	With AI	Speedup
Write 200-line Lean proof	4–8 hours	30–60 min	∼8×
Find correct Mathlib API	1–2 hours	5–15 min	∼6×
Debug type unification error	30–60 min	10–20 min	∼3×
Update all docstrings	2–3 hours	15–30 min	∼8×
Design proof architecture	2–4 hours	1–2 hours	∼2×
Choose next axiom to eliminate	30 min	15 min	∼2×

The pattern: the AI's largest speedups are in execution (3–8×), but it also provides meaningful acceleration for strategic tasks (∼2×). The AI cannot replace the human's judgment on proof architecture, but it contributes substantively by proposing concrete strategies, evaluating feasibility against the Mathlib API, and identifying when an approach requires too many new axioms.

4 AI Failure Modes

We identify four recurring failure modes:

4.1 Stale Context

The AI's context window (∼200K tokens) cannot hold the full codebase (∼20K lines of active code + Mathlib). At the start of each session, the AI must re-read relevant files, occasionally missing dependencies or using stale mental models.

Symptom: The AI writes code that references a function signature that was changed in a previous session.

Frequency: ∼1–2 times per session.

Mitigation: The AI was trained to always re-read touched files before modifying them.

4.2 API Hallucination

The AI generates plausible-sounding but non-existent Mathlib function names. For example, suggesting `Integrable.sum_le_integral_Ico` when the actual name is `AntitoneOn.inner_le_lnorm_mul_lnor`

Symptom: Compilation failure with "unknown identifier."

Frequency: ∼3–5 times per proof attempt.

Mitigation: The AI was trained to grep the Mathlib source for function names before using them, and to propose "search strategies" rather than specific API calls.

4.3 Axiom Drift

Adding a new file or import can silently pull axioms into the transitive dependency graph. The AI, not tracking axiom counts proactively, sometimes introduced unnecessary axioms.

Symptom: `#print axioms` shows unexpected axioms on the crown path.

Frequency: ∼1 time per major refactoring.

Mitigation: The human established a policy of running `#print axioms` after every structural change.

4.4 Over-Eagerness

The AI tends to produce a "working" proof quickly but without considering elegance, minimality, or future maintainability. A proof that uses 5 axioms when 2 would suffice is "correct" but not "good."

Symptom: The human reviews a proof and says "This is right but wrong."

Frequency: Persistent, requiring constant human oversight.

Mitigation: The human established a "sweep the floor" maintenance policy and reviewed all proofs for axiom minimality.

5 Comparison with Existing AI-for-Math Systems

	AlphaProof	**LeanDojo**	**Lean Copilot**	**Cathedral**
Scale	Single problem	Single lemma	Single tactic	78-file project
Autonomy	High	High	Low	Low (human-directed)
Math level	IMO	Mathlib	Mathlib	Research frontier
Novel math	Some	None	None	Collaborative
Architecture	N/A	N/A	N/A	Human-directed
LOC	∼100s	∼10s	∼1s	20,000

The Cathedral occupies a unique niche: large-scale, human-directed, operating at the research frontier. Existing AI-for-math systems excel at small-scale autonomous proving (AlphaProof) or tactic suggestion (Lean Copilot). None have been applied to a multi-file formal verification project at the scale of the Cathedral.

5.1 What Would Change the Game

For AI to move from "fast typist" to "proof strategist," the following capabilities would be needed:

1. **Long-term memory**: The ability to maintain a coherent model of a 20,000-line codebase across sessions.

2. **Axiom awareness**: Built-in tracking of which axioms a proof depends on, with optimization for minimality.

3. **Mathematical taste**: The ability to distinguish between "correct but ugly" and "correct and elegant" proofs. This is the hardest capability to define or train for.

4. **Strategic planning**: The ability to decompose a multi-week project into phases, choose proof architectures, and backtrack when an approach fails.

Of these, (1) is a systems problem, (2) is a tools problem, (3) is an open research question, and (4) is the frontier.

6 The 54% Discard Rate

The Cathedral produced 174 files, of which 96 (55%) were eventually archived. This discard rate is unusually high for software engineering but may be characteristic of formal proof search.

6.1 Why So Much Waste?

- **Exploration is cheap with AI**: Because the AI can produce a 200-line proof attempt in 30 minutes, there is little cost to trying an approach that might fail. Without the AI, each attempt would take 4–8 hours, strongly discouraging exploration.

- **Proof search is inherently exploratory**: Unlike software, where the target behavior is known, proof search explores an unknown space. Dead ends are information.

- **The archive preserves negative knowledge**: Each archived file documents an approach that doesn't work, preventing future explorers from repeating it.

6.2 Is the Discard Rate Optimal?

We conjecture that the optimal discard rate for a frontier proof project lies between 30% and 70%. Below 30%, the exploration is too conservative; above 70%, the implementation lacks focus. The Cathedral's 55% suggests a roughly balanced exploration-exploitation tradeoff, enabled by the AI's ability to make exploration cheap.

7 Lessons for AI-Assisted Mathematics

1. **AI multiplies both throughput and reach.** The Cathedral could not have been built in 23 days without AI assistance. The AI was a 5× throughput multiplier *and* expanded the space of approaches that could be explored, since the cost of trying a new proof path was minutes rather than hours.

2. **The bottleneck is taste, not skill.** The AI can write correct Lean code faster than any human. What it cannot reliably do is decide *which* correct code to write among competing alternatives. Mathematical taste—the ability to distinguish elegant from merely valid approaches—remains the human's primary contribution.

3. **Verification is the trust anchor.** In a workflow where AI generates most of the code, the Lean type-checker serves as the ultimate quality gate. The human doesn't need to verify every line—the compiler does that. The human needs to verify that the *right thing* was proved.

4. **Discard culture is healthy.** The willingness to archive 96 files without regret is essential. Sunk cost fallacy is the enemy of proof search. The AI's ability to regenerate code cheaply makes discarding psychologically easier.

5. **The collaboration is complementary, not hierarchical.** The most productive interaction pattern was genuinely collaborative: the human provided direction, the AI proposed concrete strategies, the human refined them, and the AI implemented. The AI's mathematical proposals were often adopted; its weakest contributions were in aesthetic judgment, not mathematical reasoning.

8 Conclusion

The Cathedral demonstrates that LLMs can be genuine mathematical collaborators, not merely fast typists. The collaboration produced 22,670 lines of machine-verified proof in 23 days—a pace that would be extraordinary for any team, let alone a single mathematician paired with an AI.

The AI's contributions went beyond translation. It proposed proof strategies that were adopted, designed axiom elimination campaigns, identified algebraic shortcuts, and recognized failing approaches. The human's irreplaceable contribution was taste: the judgment that selects between competing correct approaches based on elegance, minimality, and long-term architectural coherence.

The title of this paper requires revision: the AI is not merely the fast typist. It is the fast typist *and* a substantive mathematical collaborator. But it is not yet the architect. The human draws the blueprints; the AI helps draft them, builds the walls, and occasionally suggests a better window. The next cathedral will be built faster, and the AI will draw more of the plans.

References

[1] Google DeepMind, *AlphaProof and AlphaGeometry 2*, 2024.

[2] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models and the LeanDojo Benchmark*, NeurIPS 2023.

[3] S. Song et al., *Towards Large Language Models as Copilots for Theorem Proving in Lean*, 2024.

[4] A. Newell and H. Simon, *The Logic Theory Machine*, IRE Transactions on Information Theory, 1956.

[5] L. de Moura et al., *The Lean 4 Theorem Prover*, CADE-28, 2021.